

Python PRESENTATION

mouad

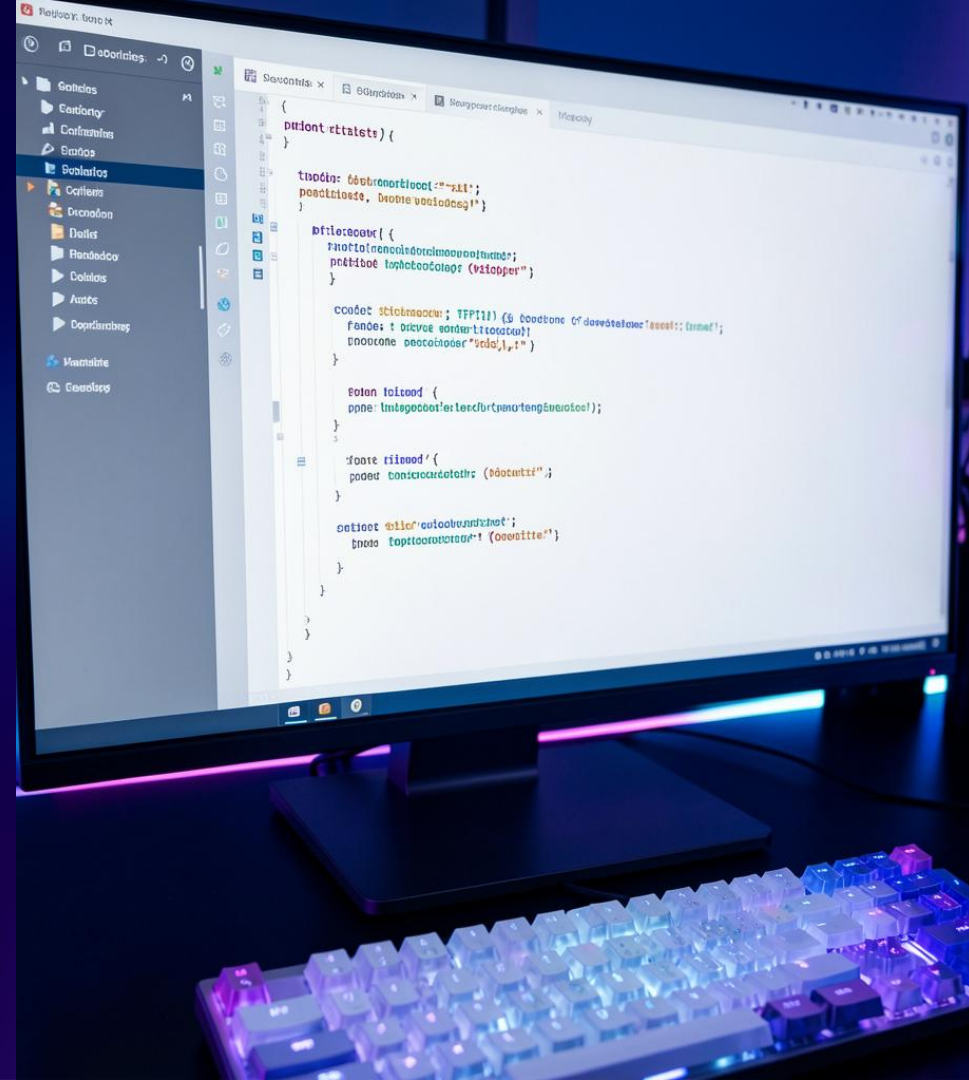
Introduction



Python est un langage de programmation interprété, lisible et polyvalent. Il favorise la *productivité* grâce à une syntaxe claire et un riche écosystème de bibliothèques. Aujourd'hui, Python est central dans la science des données, le web et l'automatisation, apprécié pour sa rapidité de prototypage et sa large communauté.

Plan de la présentation

- 1: Introduction
- 2: Définition courte et importance
- 3: Origine et auteur
- 4: Usages clés
- 5: Avantages principaux
- 6: Syntaxe de base
- 7: Exemple
- 8: Conclusions





Définition courte et importance

Python est un langage haut niveau, orienté objet et interprété, conçu pour la lisibilité.

Son adoption est portée par la **simplicité** et l'**écosystème** (bibliothèques pour data, web, IA), ce qui en fait un choix stratégique pour l'innovation et la réduction du temps de mise sur le marché.

Origine et auteur



Créé par Guido van Rossum en 1991, Python est né pour remplacer ABC, avec un accent sur la simplicité et l'extensibilité.

Depuis sa première version, il a évolué (Python 2 → Python 3), bénéficiant d'une gouvernance ouverte et d'une large adoption industrielle et académique.

Depuis sa première version, il a évolué (Python 2 → Python 3),
bénéficiant d'une gouvernance ouverte et d'une large adoption
industrielle et académique.

Usages clés

Python est largement utilisé en *data science*, développement web, automatisation et ingénierie système.

Exemples concrets : pipelines ML, API web (Django/Flask), scripts d'automatisation.

Entreprises : Google, Spotify, Netflix adoptent Python pour prototypage rapide et production.

Avantages principaux

Python offre une grande lisibilité et une courbe d'apprentissage réduite, facilitant la collaboration interdisciplinaire.

Son écosystème (bibliothèques, frameworks) et sa communauté active accélèrent l'innovation et la maintenance des projets en entreprise.

Syntaxe de base

Variables simples : `x = 10`. Conditions avec *if/else*. Boucles : *for* et *while* pour itérer.

Fonctions : `def ma_fonction(param): return valeur`. La syntaxe privilégie la clarté et réduit le code boilerplate.

Exemple



a = 10

b = 5

if a > b:

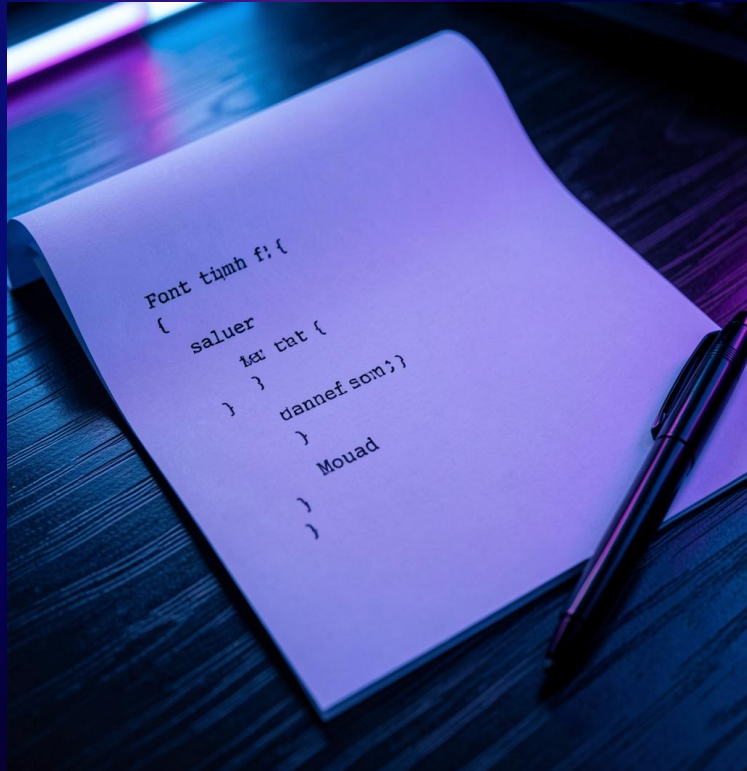
print("a est plus grand")

else:

print("b est plus grand")

a est plus grand

Conclusions



Python combine *simplicité* et puissance, adapté au prototypage et à la production.

Mon avis : Python restera central grâce à son écosystème et sa communauté. L'avenir : renforcement en IA, data engineering et automatisation d'entreprise.